

CSE5095 — Homework 2 Walkthrough

Type Safety, by Worked Analogy

How to use this. Every section below works a *different* construct than the homework, but uses *the same technique* you need. Read the worked example, then do the matching task yourself (the fill-in-the-blank skeletons in `hw2_hints.tex` pair well with this). Nothing here is an answer to a homework task. Your syllabus allows AI for understanding the material; the submitted write-up has to be yours.

The four judgments you’ll use.

- $e \text{ val}$: e is finished evaluating.
- $e \mapsto e'$: e takes one step to e' .
- $e : \tau$: e has type τ .
- Theorems: **Progress** (if $e : \tau$ then $e \text{ val}$ or $\exists e'. e \mapsto e'$) and **Preservation** (if $e : \tau$ and $e \mapsto e'$ then $e' : \tau$).

The one distinction to keep in your head.

- A *search rule* has a *step* judgment as a premise (e.g. StepSearchAddLeft: “if $e_1 \mapsto e'_1$ then $e_1 + e_2 \mapsto e'_1 + e_2$ ”). It just says *where* to evaluate next.
- A *primitive / reduction rule* (StepAdd, StepCat, ...) does the actual computation. It has no step premises, though it may have side conditions like $n_2 \neq 0$.

In proofs, reduction-rule cases use **inversion**. Search-rule cases use **the induction hypothesis**.

Part A: Booleans (Tasks 1–6)

Worked construct: pairs. We extend the class language E with

$$\tau ::= \dots \mid \tau_1 \times \tau_2 \qquad e ::= \dots \mid (e_1, e_2) \mid \text{fst } e$$

Intended meaning: evaluate both components left to right; `fst` of a finished pair returns the first component.

A1. Writing dynamic rules

(technique for Task 1)

Recipe.

1. *Value rules:* which finished forms of the new syntax are done?
2. *Reduction rules:* once the sub-expressions you care about are values, what does the construct compute to?
3. *Search rules:* for each sub-expression that must be evaluated *before* reducing, add one rule that steps it and leaves everything else alone. Do **not** add search rules for sub-expressions that shouldn't be evaluated yet.

Worked. A pair is a value when both parts are values. This value rule *has premises*, unlike ValNum:

$$\frac{e_1 \text{ val} \quad e_2 \text{ val}}{(e_1, e_2) \text{ val}} \text{ (VALPAIR)}$$

Reduction: `fst` of a finished pair gives the first component:

$$\frac{e_1 \text{ val} \quad e_2 \text{ val}}{\text{fst } (e_1, e_2) \mapsto e_1} \text{ (STEPFST)}$$

Search: we need to evaluate the left component, then the right one (only once the left is a value), and the argument of `fst` :

$$\frac{e_1 \mapsto e'_1}{(e_1, e_2) \mapsto (e'_1, e_2)} \text{ (STEPSEARCHPAIRL)} \quad \frac{e_1 \text{ val} \quad e_2 \mapsto e'_2}{(e_1, e_2) \mapsto (e_1, e'_2)} \text{ (STEPSEARCHPAIRR)}$$
$$\frac{e \mapsto e'}{\text{fst } e \mapsto \text{fst } e'} \text{ (STEPSEARCHFST)}$$

Sanity checks to run on your own rules.

- For every non-value expression, is there *exactly one* rule that applies? (No rule gives a stuck expression; two rules means your semantics isn't deterministic.)
- Does anything get evaluated that the English description says *shouldn't* be? For if, reread the assignment's if $x = 0$ then 0 else $42/x$ remark, and ask which sub-expressions deserve a search rule.

Your turn: Task 1. Apply the recipe to `true`, `false`, and `if`. Target count from the assignment: 2 value rules, 3 step rules, exactly 1 of which is a search rule.

A2. Evaluating step by step

(technique for Task 2)

Recipe. At each line, find the *innermost-leftmost* piece that a reduction rule can fire on. Then name the stack of search rules that “reaches in” to it, from the outside in.

Worked. Evaluate $\text{fst } (\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"})$:

$$\begin{array}{ll}
& \text{fst } (\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"}) \\
\mapsto & \text{fst } (\bar{3}, \text{"a"}^{\wedge}\text{"b"}) \quad \text{StepSearchFst} \leftarrow \text{StepSearchPairL} \leftarrow \text{StepAdd} \\
\mapsto & \text{fst } (\bar{3}, \text{"ab"}) \quad \text{StepSearchFst} \leftarrow \text{StepSearchPairR} \leftarrow \text{StepCat} \\
\mapsto & \bar{3} \quad \text{StepFst}
\end{array}$$

$\bar{3}$ is a value by ValNum, so we stop. The pair $(\bar{3}, \text{"ab"})$ wasn't the end, because it sits inside fst .

The " $\leftarrow$ " stack is shorthand for a real derivation tree. Here is the first step written out in full. Put one like this in your write-up if you want to be extra rigorous:

$$\frac{\frac{\overline{\bar{1} + \bar{2} \mapsto \bar{3}} \text{ (STEPADD)}}{\overline{(\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"}) \mapsto (\bar{3}, \text{"a"}^{\wedge}\text{"b"})} \text{ (STEPSEARCHPAIRL)}}}{\overline{\text{fst } (\bar{1} + \bar{2}, \text{"a"}^{\wedge}\text{"b"}) \mapsto \text{fst } (\bar{3}, \text{"a"}^{\wedge}\text{"b"})} \text{ (STEPSEARCHFST)}}$$

The second step also needs a leaf $\bar{3}$ val (ValNum), because StepSearchPairR has that premise.

Your turn: Task 2. Your expression has three nesting levels: if $\supset = \supset +$. Your first step's stack has one rule per level.

A3. Typing derivations

(technique for Task 3)

Typing rules for pairs:

$$\frac{e_1 : \tau_1 \quad e_2 : \tau_2}{(e_1, e_2) : \tau_1 \times \tau_2} \text{ (TYPEPAIR)} \qquad \frac{e : \tau_1 \times \tau_2}{\text{fst } e : \tau_1} \text{ (TYPEFST)}$$

Recipe. Write the goal at the bottom. Look at the *outermost* syntax to pick the rule, write its premises above the line, and repeat until every branch ends in an axiom (a rule with no premises).

Worked. $\text{fst } (\bar{1} + \bar{2}, \text{"hi"}) : \text{int}$

$$\frac{\frac{\overline{\bar{1} : \text{int}} \text{ (TN)} \quad \overline{\bar{2} : \text{int}} \text{ (TN)}}{\overline{\bar{1} + \bar{2} : \text{int}} \text{ (TA)}} \quad \overline{\text{"hi"} : \text{string}} \text{ (TS)}}{\overline{(\bar{1} + \bar{2}, \text{"hi"}) : \text{int} \times \text{string}} \text{ (TPAIR)}}}{\overline{\text{fst } (\bar{1} + \bar{2}, \text{"hi"}) : \text{int}} \text{ (TFST)}}$$

When a rule like TypeFst lets you choose (here, any τ_2), the sub-derivation decides it. With if, TypeIf forces both branches to have the same τ , and τ is also the type of the whole expression.

Your turn: Task 3. The outermost rule is TypeIf with *three* premises. The condition's sub-tree looks just like the $\bar{1} + \bar{2}$ sub-tree above, with one more rule (TypeEq) underneath.

A4. Preservation cases

(technique for Task 4)

Structure: induction on the derivation of $e \mapsto e'$, one case per step rule. Each case follows this template:

1. “Then $e = \dots$ and $e' = \dots$ ” (read off the rule).
2. Invert the typing rule for e ’s outermost form, to learn the types of its pieces.
3. *Reduction rule:* e' is one of those pieces (or a new literal), so its type is already known, or comes from an axiom.
Search rule: apply the IH to the premise $e_i \mapsto e'_i$, then re-apply the same typing rule.

Worked: Case StepFst. Then $e = \text{fst } (e_1, e_2)$ and $e' = e_1$, with e_1 **val** and e_2 **val**. We know $e : \tau$. The only rule concluding $\text{fst} \cdot : \tau$ is TypeFst, so by inversion $(e_1, e_2) : \tau \times \tau_2$ for some τ_2 . The only rule concluding a pair’s type is TypePair, so inverting again gives $e_1 : \tau$ and $e_2 : \tau_2$. So $e' = e_1 : \tau$. $\square$

Worked: Case StepSearchFst. Then $e = \text{fst } e_0$ and $e' = \text{fst } e'_0$ with $e_0 \mapsto e'_0$. By inversion on TypeFst, $e_0 : \tau \times \tau_2$ for some τ_2 . By the induction hypothesis on $e_0 \mapsto e'_0$, $e'_0 : \tau \times \tau_2$. By TypeFst, $e' = \text{fst } e'_0 : \tau$. $\square$

Watch out. In the search case, the IH is about the *sub-expression*’s type ($\tau \times \tau_2$ here), not τ . Say which type you’re applying it at.

Your turn: Task 4. Three cases, one per Task 1 step rule. Your two reduction cases each need *one* inversion (on TypeIf).

A5. Canonical Forms

(technique for Task 5)

Recipe. Canonical forms answers: “which value rules could possibly have produced a value of this type?” List those shapes.

Worked. If e **val** and $e : \tau_1 \times \tau_2$, then $e = (e_1, e_2)$ for some e_1, e_2 with e_1 **val** and e_2 **val**. (ValNum and ValString produce things of type **int** and **string**, so only ValPair can produce a value of a product type.)

Your turn: Task 5. Which of your Task 1 value rules produce something of type **bool**? A type may have more than one canonical form, and that’s fine. It just means a case split later.

A6. Progress cases

(technique for Task 6)

Structure: induction on the derivation of $e : \tau$. Template for a rule with sub-expressions:

1. Invert to get the sub-expression typings (the premises of the rule).
2. Apply the IH to the sub-expression that gets evaluated first. You get two cases: it’s a value, or it steps.

3. *Steps*: use the search rule. *Value*: use Canonical Forms to get its exact shape, then use the reduction rule for that shape.

Worked: Case TypeFst. Then $e = \text{fst } e_0$ with $e_0 : \tau \times \tau_2$. By the IH, e_0 **val** or $e_0 \mapsto e'_0$.

- Suppose e_0 **val**. By Canonical Forms (product case), $e_0 = (e_1, e_2)$ with e_1 **val** and e_2 **val**. By StepFst, $e \mapsto e_1$.
- Suppose $e_0 \mapsto e'_0$. By StepSearchFst, $e \mapsto \text{fst } e'_0$.

□

Worked: Case TypePair (for the value-rule side). Here *both* components get evaluated, so we need the IH on both, nested exactly like the class's TypeAdd case. If both are values, e **val** by ValPair (not a step!). If e_1 is a value and e_2 steps, use StepSearchPairR. If e_1 steps, use StepSearchPairL.

Your turn: Task 6. TypeTrue and TypeFalse are one-liners (like TypeNum). For TypeIf, how many sub-expressions does your semantics evaluate first? That tells you how many IHs you need. In the value case, Canonical Forms gives you *two* shapes, so you get two sub-cases.

Part B: Division by zero (Tasks 7–9)

Worked system: SQ, a language that prevents *square roots of negatives*. Types $\tau ::= \text{nn} \mid \text{any}$ (“definitely non-negative” vs. “any integer”). Expressions: $e ::= \bar{n} \mid e + e \mid e - e \mid e * e \mid \text{sqrt } e$, with arithmetic dynamics like EZ’s, plus

$$\frac{n \geq 0}{\text{sqrt } \bar{n} \mapsto \lfloor \sqrt{n} \rfloor} \text{ (STEPSQRT)} \quad \frac{e \mapsto e'}{\text{sqrt } e \mapsto \text{sqrt } e'} \text{ (STEPSEARCHSQRT)}$$

So $\text{sqrt } \overline{-4}$ is stuck, just like $\bar{1}/\bar{0}$ in EZ. As in EZ, if $e : \text{nn}$ then $e : \text{any}$, and Canonical Forms says nn-values are $\bar{n}$ with $n \geq 0$.

B1. Designing refinement rules (technique for Task 7)

Recipe (the whole trick). For each blank, try the *stronger* type first.

- To show the stronger type is **unsafe**, find *one* counterexample: inputs satisfying the premises where the output breaks the property.
- To show it’s **safe**, give an arithmetic argument that works for *all* such inputs.

Worked. Base rules: $\frac{}{\bar{n} : \text{any}} \text{ (TYPENUM)}$ and $\frac{n \geq 0}{\bar{n} : \text{nn}} \text{ (TYPENUMNN)}$.

- $e_1 : \text{nn}, e_2 : \text{nn} \Rightarrow e_1 + e_2 : ?$ A sum of two non-negatives is ≥ 0 for *all* inputs. Safe, so nn.
- $e_1 : \text{nn}, e_2 : \text{nn} \Rightarrow e_1 - e_2 : ?$ Counterexample: $0 - 1 = -1$. So only any.
- $e_1 : \text{nn}, e_2 : \text{nn} \Rightarrow e_1 * e_2 : ?$ A product of non-negatives is ≥ 0 . So nn.
- $\text{sqrt } e$: StepSqrt’s side condition is $n \geq 0$, so the premise must be $e : \text{nn}$. Result: $\lfloor \sqrt{n} \rfloor \geq 0$ always. So nn.

Plus any-versions ($e_1 : \text{any}, e_2 : \text{any} \Rightarrow e_1 \text{ op } e_2 : \text{any}$) so that mixed programs still type check.

Your turn: Task 7. Same drill for EZ’s blanks. The assignment reminds you integers can be negative, and division truncates. Test $\bar{1}/\bar{2}$ before you decide the result type of TypeDiv.

B2. Progress with a side condition (technique for Task 8, Progress)

Worked: Case TypeSqrt. Then $e = \text{sqrt } e_0$, $\tau = \text{nn}$, and $e_0 : \text{nn}$. By the IH, $e_0 \text{ val}$ or $e_0 \mapsto e'_0$.

- Suppose $e_0 \text{ val}$. By Canonical Forms (**the nn case**), $e_0 = \bar{n}$ with $n \geq 0$. That’s exactly StepSqrt’s side condition, so $e \mapsto \lfloor \sqrt{n} \rfloor$.
- Suppose $e_0 \mapsto e'_0$. By StepSearchSqrt, $e \mapsto \text{sqrt } e'_0$.

□

The point: the *type* of the argument is what gives you the side condition, through Canonical Forms. That’s the whole reason the type system exists. TypeDiv has two sub-expressions, so you’ll nest two IHs (like the class’s TypeAdd case) and use the nz case of Canonical Forms only on the one where it matters.

B3. Preservation when inversion gives two rules (Task 8 & Bonus)

Worked: Case StepAdd in SQ. Then $e = \overline{n_1} + \overline{n_2}$ and $e' = \overline{n_1 + n_2}$. Two typing rules conclude a sum, so inversion gives two sub-cases:

- *Via TypeAddNN.* Then $\tau = \text{nn}$ and $\overline{n_1} : \text{nn}$, $\overline{n_2} : \text{nn}$. Inverting again (the only rule typing a numeral nn is TypeNumNN): $n_1 \geq 0$ and $n_2 \geq 0$. So $n_1 + n_2 \geq 0$, and by TypeNumNN, $e' : \text{nn}$.
- *Via TypeAddAny.* Then $\tau = \text{any}$. By TypeNum, $e' : \text{any}$.

□

Your turn. In EZ, only *one* rule concludes a sum, a difference, or a quotient, so StepAdd, StepSub, and StepDiv each have one case. Ask yourself what τ inversion forces, and which base typing rule retypes the resulting numeral. The Bonus (StepMul) is the two-rule situation shown above.

B4. Finding the type system’s blind spot (technique for Task 9)

Recipe. Find a rule that *throws away information* (its conclusion is weaker than what’s actually true for some inputs). Build an expression whose runtime value is fine, but only because of the information that got thrown away. Then propose extra information to track, and explain why the fix still blocks the bad programs.

Worked. In SQ, consider $\text{sqrt}(\overline{-2} * \overline{-3})$. At runtime, $(-2)(-3) = 6 \geq 0$, so it steps to $\overline{2}$ with no error. But $\overline{-2}$ and $\overline{-3}$ can only be typed any , so the product is any and TypeSqrt rejects it. The information “both factors are negative” was never recorded.

Fix: add a type np (“non-positive”) with

$$\frac{n \leq 0}{\overline{n} : \text{np}} (\text{TYPE_NUMNP}) \quad \frac{e_1 : \text{np} \quad e_2 : \text{np}}{e_1 * e_2 : \text{nn}} (\text{TYPE_MULNP})$$

Still safe: each new rule is backed by a true arithmetic fact (a product of two non-positives is ≥ 0), so nothing that evaluates to a negative can be typed nn . $\text{sqrt}(\overline{-4})$ is still rejected.

Your turn: Task 9. Look at which EZ rules only ever conclude mz . The assignment says your fix doesn’t need to be complete or practical. One or two sentences of justification is fine.

LaTeX quick reference (this homework’s macros.tex)

- $\text{\Rule{Name}{prem1} \ \ \ \text{prem2}{conclusion}}$: a labeled rule. Use $\text{\strut}$ for no premises.
- Nest a $\text{\Rule}$ inside a premise to build derivation trees. Wrap big ones in $\text{\small ...}$ or $\text{\scriptsize}$.
- Evaluation traces: an array with columns $\text{\{r l l\}}$ ($\text{\mapsto}$, expression, rule names).
- Overlined numerals: $\text{\overline{3}}$.

Final checklist

- Task 1: 5 rules, exactly one search rule, no rule evaluates an un-chosen branch.
- Task 2: every step names its rule stack. Stop at a value.
- Task 3: every leaf is an axiom. Both branches get the same type.
- Tasks 4/6/8: say what you're inducting on. Each case names the rule it inverts on and the rule it concludes with.
- Task 8 Progress (TypeDiv): the $n_2 \neq 0$ side condition is justified by Canonical Forms, not assumed.
- Task 9: an expression, why it fails to type check, a fix, and why the fix is still safe.
- Task 10: don't forget it!